

Network Programming 2026 - Assignment #2

Course: Network Programming

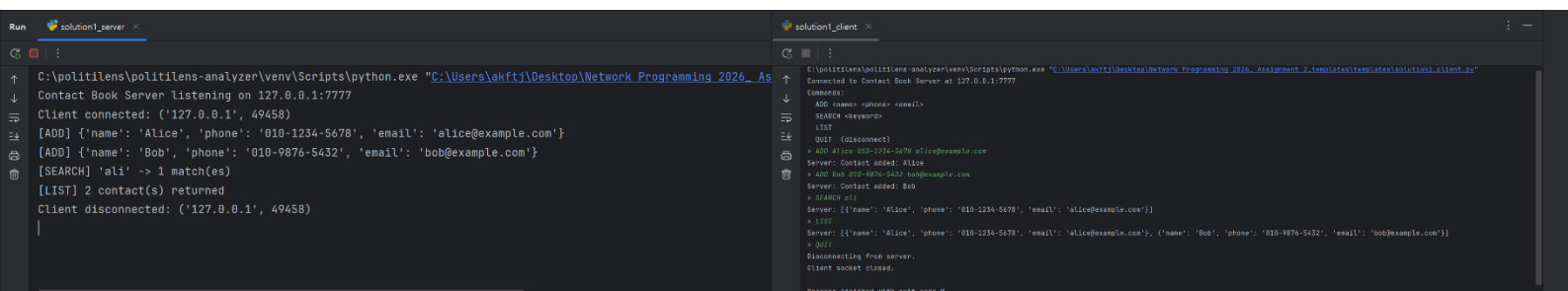
Assignment: Assignment #2 (Chapters 4, 5, 6)

Name: 홍성주

Student ID: 20221528

Question 1. Structured Contact Book over TCP

1. Screenshots



```
Run solution_server x
C:\politiLens\politiLens-analyzer\venv\Scripts\python.exe "C:\Users\akftj\Desktop\Network Programming 2026_Assignment_2\template\template\solution_server.py"
Contact Book Server listening on 127.0.0.1:7777
Client connected: ('127.0.0.1', 49458)
[ADD] {'name': 'Alice', 'phone': '010-1234-5678', 'email': 'alice@example.com'}
[ADD] {'name': 'Bob', 'phone': '010-9876-5432', 'email': 'bob@example.com'}
[SEARCH] 'ali' -> 1 match(es)
[LIST] 2 contact(s) returned
Client disconnected: ('127.0.0.1', 49458)

solution_client x
C:\politiLens\politiLens-analyzer\venv\Scripts\python.exe "C:\Users\akftj\Desktop\Network Programming 2026_Assignment_2\template\template\solution_client.py"
Connected to Contact Book Server at 127.0.0.1:7777
Commands:
> ADD <name> <phone> <email>
SEARCH <keyword>
LIST
QUIT (Disconnect)
> ADD Alice 010-1234-5678 alice@example.com
Server: Contact added: Alice
> ADD Bob 010-9876-5432 bob@example.com
Server: Contact added: Bob
> SEARCH ali
Server: [{'name': 'Alice', 'phone': '010-1234-5678', 'email': 'alice@example.com'}]
> LIST
Server: [({'name': 'Alice', 'phone': '010-1234-5678', 'email': 'alice@example.com'}, {'name': 'Bob', 'phone': '010-9876-5432', 'email': 'bob@example.com'})]
> QUIT
Disconnecting from server.
Client socket closed.
Process finished with exit code 0
```

2. My Code Logic

For this problem, I used the pickle and struct modules. At first, I thought `recv()` would receive the whole message at once, but it didn't, so I needed a way to separate messages.

Since TCP is just a stream of bytes, I added a 4-byte header using `struct.pack('>I', length)` before the actual data (big-endian unsigned int). On the receiving side, I read exactly 4 bytes first in a while loop to get

the length, then read the rest of the payload in another loop until the exact byte count arrived. This way, partial `recv()` calls do not corrupt the message boundary. I also used `sendall()` so the kernel is forced to flush the whole buffer.

The server handles `ADD`, `SEARCH`, and `LIST` commands and stores the contacts in memory. `SEARCH` does a case-insensitive substring match on the name field. I set `SO_REUSEADDR` on the listening socket so I can restart the server right after stopping it without an "Address already in use" error.

3. How I Handled Errors

While testing, the program sometimes crashed when I entered a wrong command. To fix this, I made `parse_input()` return `None` when the input format is incorrect (for example, when the email is missing), and the main loop prints a usage hint instead of sending garbage to the server.

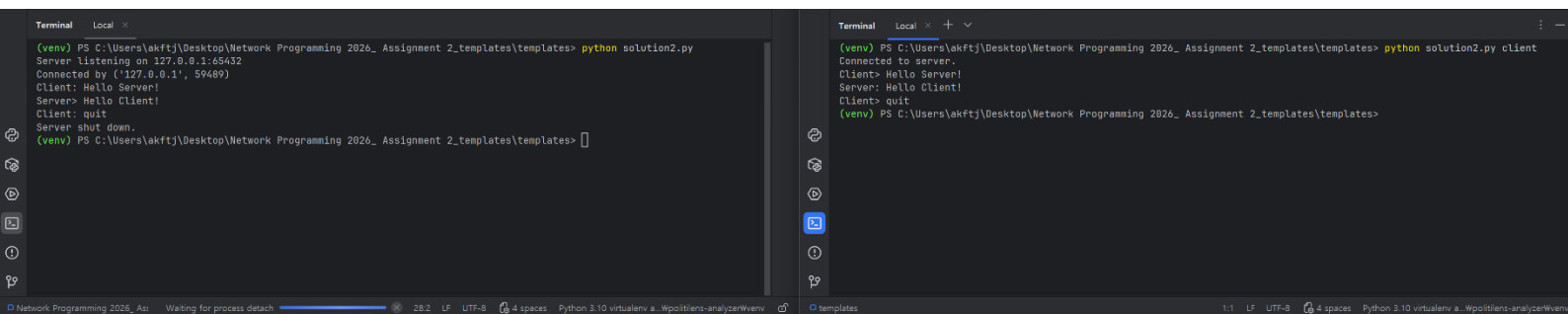
On the server side, I added `try/except` blocks that handle `ConnectionResetError`, `BrokenPipeError`, and `EOFError` so the server keeps running after a client suddenly disconnects. `pickle.UnpicklingError` is caught separately in case a malformed payload arrives, and a generic

Exception handler guarantees that no single bad client can ever take the server down. Every client socket is closed inside a finally block.

On the client side, I separately catch `ConnectionRefusedError` ("Is the server running?"), `KeyboardInterrupt`, and `EOFError` so the user always sees a clear message instead of a raw traceback.

Question 2. Debug the Broken Chat Server

1. Screenshots



The image shows two side-by-side terminal windows. The left window shows the server's execution: it starts by listening on 127.0.0.1:65432, receives a connection from 127.0.0.1:59489, prints 'Client: Hello Server!', 'Server: Hello Client!', and 'Client: quit', and finally prints 'Server shut down.' before returning to the prompt. The right window shows the client's execution: it connects to the server, prints 'Connected to server.', 'Client: Hello Server!', 'Server: Hello Client!', and 'Client: quit', and then returns to the prompt.

```
(venv) PS C:\Users\akftj\Desktop\Network Programming 2026_ Assignment 2_templates\templates> python solution2.py
Server listening on 127.0.0.1:65432
Connected by ('127.0.0.1', 59489)
Client: Hello Server!
Server: Hello Client!
Client: quit
Server shut down.
(venv) PS C:\Users\akftj\Desktop\Network Programming 2026_ Assignment 2_templates\templates>

(venv) PS C:\Users\akftj\Desktop\Network Programming 2026_ Assignment 2_templates\templates> python solution2.py cClient
Connected to server.
Client: Hello Server!
Server: Hello Client!
Client: quit
(venv) PS C:\Users\akftj\Desktop\Network Programming 2026_ Assignment 2_templates\templates>
```

2. My Code Logic

I found FIVE distinct bugs in the given template and marked each fix with a "# FIX N:" comment in solution2.py.

FIX 1 — The server socket was created with `SOCK_DGRAM`, which is UDP.

UDP sockets do not support `listen()/accept()`, so I changed it to

`SOCK_STREAM` (TCP).

FIX 2 — The server's reply path called `conn.send(reply)`, but `socket.send()`

only accepts bytes. I added `.encode("utf-8")` to convert the str reply to

bytes. Without this, the program raises `TypeError`.

FIX 3 — After the server typed "quit" to end the chat, the loop went

straight back to `recv()` and blocked forever. I added

``if reply.lower() == "quit": break`` so the server can also end the chat

from its own side.

FIX 4 — The client displayed incoming data with `f"Server: {data}"`, which

prints the bytes literal `b'Hello'` instead of a readable string. I added

`data.decode("utf-8")` before printing.

FIX 5 — The client only broke the loop on its own "quit"; when the server

sent "quit" the client kept prompting for another message. I added

``if message.lower() == "quit": break`` on the client side so both sides can

end the chat symmetrically.

3. How I Handled Errors

Most of the problems in this part were logical/API errors rather than

runtime exceptions. Fixing the socket type (FIX 1) allowed the server to

start at all; fixing the encode/decode pair (FIX 2, FIX 4) made the bytes

API work both ways; fixing the two "quit" conditions (FIX 3, FIX 5) made

the chat ending symmetric so neither side deadlocks waiting for the

other. After these five fixes, the chat no longer raises exceptions

under normal use.

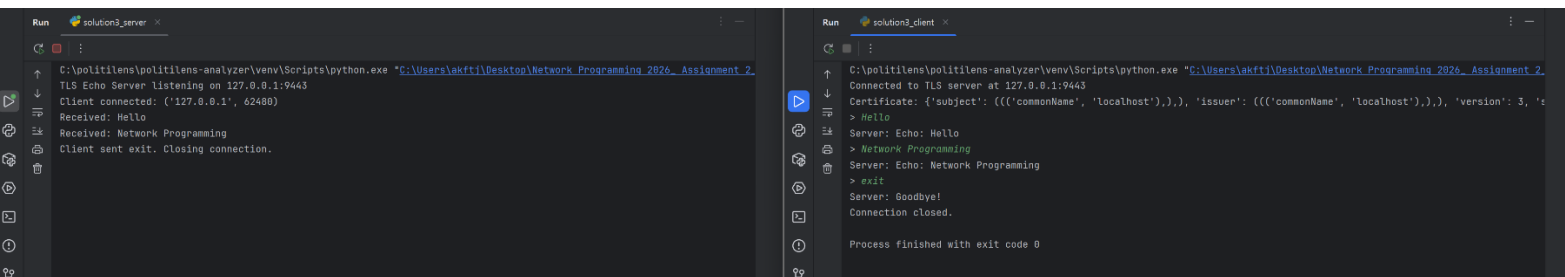
Question 3. TLS-Secured Echo Service

1. Screenshots

[illegible]

```
(venv) PS C:\Users\akftj\Desktop\Network Programming 2026_ Assignment 2_templates\templates> openssl x509 -in server.crt -noout -text | Select-String "Subject:|Issuer:|DNS:|IP Address:"
Issuer: CN=localhost
Subject: CN=localhost
IP Address:127.0.0.1, DNS:localhost

(venv) PS C:\Users\akftj\Desktop\Network Programming 2026_ Assignment 2_templates\templates>
```



```
Run solution3_server
C:\poltilens\poltilens-analyzer\venv\Scripts\python.exe "C:\Users\akftj\Desktop\Network Programming 2026_ Assignment 2..."
TLS Echo Server listening on 127.0.0.1:9443
Client connected: ('127.0.0.1', 62480)
Received: Hello
Received: Network Programming
Client sent exit. Closing connection.

Run solution3_client
C:\poltilens\poltilens-analyzer\venv\Scripts\python.exe "C:\Users\akftj\Desktop\Network Programming 2026_ Assignment 2..."
Connected to TLS server at 127.0.0.1:9443
Certificate: {'subject': (('commonName', 'localhost'),), 'issuer': (('commonName', 'localhost'),), 'version': 3, 'serialNumber': 1}
> Hello
Server: Echo: Hello
> Network Programming
Server: Echo: Network Programming
> exit
Server: Goodbye!
Connection closed.
Process finished with exit code 0
```

2. My Code Logic

At first, I had trouble with certificate verification. I generated a self-signed certificate using OpenSSL with the following one-liner:

```
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt \
-days 365 -nodes \
-subj "/CN=localhost" \
-addext "subjectAltName=IP:127.0.0.1,DNS:localhost"
```

The SAN entry for IP:127.0.0.1 is essential because Python 3.7+ no longer falls back to the certificate's Common Name for hostname verification.

Without the SAN, `wrap_socket()` raises

`ssl.CertificateError: hostname '127.0.0.1' doesn't match.`

On the server side, I built an SSL context with

`ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)` and loaded the certificate and key files via `load_cert_chain()`. Every accepted TCP socket is wrapped with `ctx.wrap_socket(raw, server_side=True)`.

On the client side, the spec forbids disabling verification, so I used

`ssl.create_default_context(ssl.Purpose.SERVER_AUTH, cafile=CERTFILE)`.

`create_default_context()` automatically sets `check_hostname=True` and `verify_mode=CERT_REQUIRED`, and passing `CERTFILE` as the CA is what lets the self-signed certificate validate. I then called `ctx.wrap_socket(raw, server_hostname=HOST)` so hostname verification runs against the SAN. Right after the handshake, I print `tls.getpeercert()` — the spec's expected output format shows this dict, and printing it also proves TLS is really in use.

3. How I Handled Errors

If a non-TLS client tries to connect, `wrap_socket()` raises `ssl.SSLError`.

I catch it in the server's accept loop, log it, and continue — one bad

handshake never kills the server, and the accept loop immediately serves

the next client. `ConnectionResetError` and `BrokenPipeError` during `recv()` or `sendall()` are caught per-client for the same reason.

On the client, I separately catch `ConnectionRefusedError` ("Server not reachable"), `ssl.SSLError` ("TLS handshake failed"), and `OSError` so the user gets a clear message for each failure class instead of a stack trace.

For closing, I call `tls.shutdown(socket.SHUT_RDWR)` before `tls.close()` so the TLS `close_notify` alert is sent and the TCP FIN follows cleanly; both are wrapped in `try/except OSError` in case the peer already dropped the connection. "Connection closed." is only printed when a TLS session actually existed, so failed-connection output is not misleading.